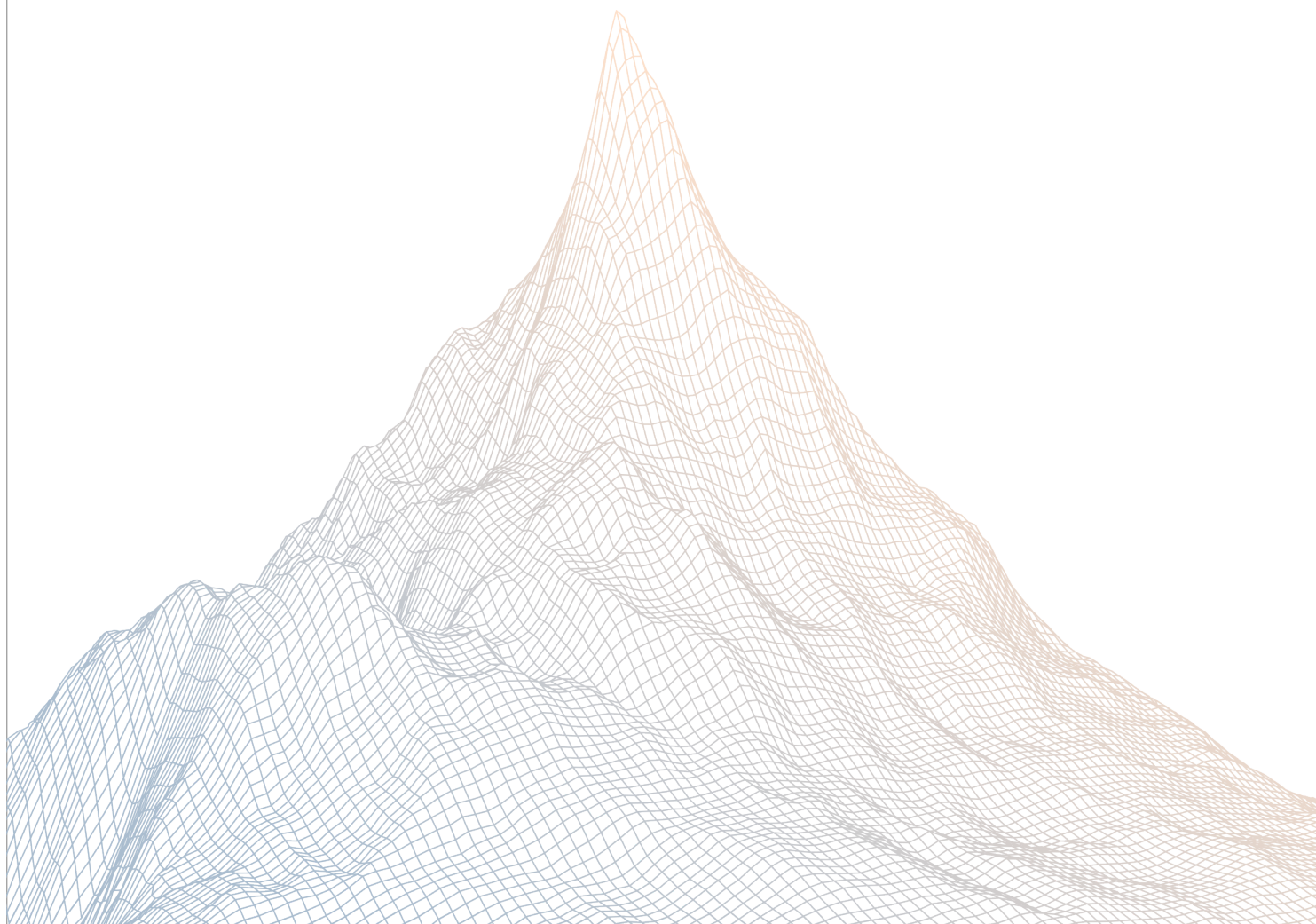


HyperLink

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

May 7th to May 13th, 2026

AUDITED BY:

adriro
ether_sky

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About HyperLink	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	6
4	Findings	8
4.1	High Risk	9
4.2	Medium Risk	14
4.3	Low Risk	24
4.4	Informational	33

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About HyperLink

Privacy-preserving prime broker protocol for Hyperliquid using AWS Nitro Enclaves with HyperEVM smart contracts managing deposits, withdrawals, and state verification. Also includes hIHYPE, a liquid staking token for HYPE.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts
Repository	https://github.com/hyperlink-xyz/contracts.git
Commit Hash	c7891fbd7d4263fdfa1763a41269d981a1d37463
Files	HyperlinkTokenRelay.sol 1st/Base.sol 1st/HLHYPE.sol 1st/StakingVault.sol 1st/StakingVaultManager.sol 1st/libraries/Converters.sol 1st/libraries/StructuredLinkedList.sol

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/hyperlink-xyz/contracts.git
-------------------	---

Commit Hash	97a700476392aa6ab344dd68adb678067514a73a
--------------------	--

Files	HyperlinkTokenRelay.sol 1st/Base.sol 1st/HLHYPE.sol 1st/StakingVault.sol 1st/StakingVaultManager.sol 1st/libraries/Converters.sol 1st/libraries/StructuredLinkedList.sol
--------------	--

2.3 Audit Timeline

May 7, 2026	Audit start
May 13, 2026	Audit end
May 22, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	3
Medium Risk	5
Low Risk	6
Informational	17
Total Issues	31

3

Findings Summary

ID	Description	Status
H-1	<code>_requireCoreSpotBalance()</code> over-reserves LST withdrawal HYPE during the 7-day unbond	Resolved
H-2	<code>stakingAccountBalance()</code> accounting is incorrect due to Core/EVM async state	Resolved
H-3	<code>withdrawStake()</code> does not validate the validator's available delegation	Resolved
M-1	The unstaking amount can become smaller due to rounding	Resolved
M-2	<code>verifyOrSyncTotalStakedHype()</code> can desync <code>totalStakedHype</code> from <code>totalReservedLstWithdrawalHype</code>	Resolved
M-3	The token relay should only be allowed to delegate to the manager's validator	Resolved
M-4	The <code>verifyOrSyncTotalStakedHype</code> and <code>stakeFromSpot</code> functions should not be called within the same block	Resolved
M-5	Stale shadow staking balance can overstate the LST exchange rate	Resolved
L-1	<code>_validateAccess()</code> has unclear and inconsistent semantics	Resolved
L-2	<code>withdrawWithAuthorization()</code> quotes fees against the net amount instead of the gross	Acknowledged
L-3	The exchange rate should decrease when a slashing is applied	Resolved
L-4	There should be a fallback mechanism in case switching to a new validator fails	Resolved
L-5	The system should not be able to unpause while in emergency mode	Resolved
L-6	Mutable claim buffer can reopen the slash window after withdrawals are claimed	Resolved
I-1	There is an unnecessary conditional check in <code>setMinimumStakeBalance</code> function	Resolved
I-2	There is a incorrect comment in <code>emergencyStakingWithdraw</code> function	Resolved

ID	Description	Status
I-3	The exchange rate can change significantly when the admin reconciles totalReservedLstWithdrawalHype	Resolved
I-4	There is an unnecessary check in the deposit function	Resolved
I-5	The minimum withdrawal amount can be calculated using the exchange rate of the current batch	Resolved
I-6	There is an unnecessary check in the processBatch function	Resolved
I-7	Dust accounting overflow check in _disburseFromCore() is overly conservative	Resolved
I-8	Withdrawal recipient is not checked against the protocol registry	Resolved
I-9	addValidator() and removeValidator() do not emit events	Resolved
I-10	Redundant whenNotPaused modifier on HLHYPE.mint()/burn()	Resolved
I-11	transferHypeToCore() can lose dust by not truncating to 8 decimals	Resolved
I-12	HYPE token id is duplicated instead of using HLConstants.hypeTokenIndex()	Resolved
I-13	EVM accounting can diverge from Core state on async failures	Resolved
I-14	usedAuthorizations mapping is redundant with the per-user nonce	Resolved
I-15	_setTargetPaused() can revert during emergency actions	Resolved
I-16	exitWithCommitment() unnecessarily increments the withdrawal nonce	Resolved
I-17	cumulativeExitWithdrawn conflates normal and emergency withdrawals	Resolved

4

Findings

4.1 High Risk

A total of 3 high risk findings were identified.

[H-1] `_requireCoreSpotBalance()` over-reserves LST withdrawal HYPE during the 7-day unbond

SEVERITY: High

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: High

Target

- [HyperlinkTokenRelay.sol](#)

Description:

When `_requireCoreSpotBalance()` is called with `includeReservedLstHype = true` (i.e. for normal `withdrawWithAuthorization()` flows), it requires:

```
availableWei ≥ requiredWei + totalReservedLstWithdrawalHype /  
HYPE_CORE_TO_EVM_SCALE
```

`totalReservedLstWithdrawalHype` is incremented as soon as `withdrawStake()` runs, but the corresponding HYPE only lands in the relay's spot account 7 days later (after the unbonding period and `claimUnstakedToVault()`). During this 7-day window the reserve is included in `required` even though the tokens are not yet in spot. Any normal user withdrawal that needs to dip into the relay's spot balance will revert with `InsufficientCoreSpotBalance` even when there is more than enough liquidity to cover both the user withdrawal and the actually-claimable LST reserve.

This is a denial of service on user withdrawals that scales with the size of pending LST unstakes: every time the operator triggers `finalizeBatch()` and increments `totalReservedLstWithdrawalHype`, the available withdrawal liquidity for non-LST users drops by that amount for ~7 days.

Recommendations:

Subtract the live `delegatorSummary.totalPendingWithdrawal` from `totalReservedLstWithdrawalHype` before adding it to `required`. `totalPendingWithdrawal`

is the unstake amount still inside the 7-day lock and not yet credited to the relay's spot, so the difference is the matured-but-unclaimed reserve that actually competes with normal withdrawals for spot liquidity. For example:

```
uint256 reserved = $.totalReservedLstWithdrawalHype /  
    HYPE_CORE_TO_EVM_SCALE;  
uint256 pending =  
    PrecompileLib.delegatorSummary(address(this)).totalPendingWithdrawal;  
required += reserved > pending ? reserved - pending : 0;
```

HyperLink: Resolved with [PR-58](#).

Zenith: Verified.

[H-2] `stakingAccountBalance()` accounting is incorrect due to Core/EVM async state

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: High

Target

- [StakingVaultManager.sol](#)

Description:

`stakingAccountBalance()` computes the LST backing as:

```
backing = delegatorSummary.delegated + delegatorSummary.undelegated +  
totalReservedLstWithdrawalHype
```

Both `delegated/undelegated` (read from the `DelegatorSummary` precompile) and `totalReservedLstWithdrawalHype` (an EVM-side counter mutated by `tokenRelay`) reference the same underlying HYPE while Core operations are still in flight, leading to systematic over- and under-counting:

- When `withdrawStake()` runs, `totalReservedLstWithdrawalHype` is incremented immediately, but the validator's `delegated` amount only decreases after unstaking. During that window the same HYPE is counted in both `delegated` and `totalReservedLstWithdrawalHype`, inflating backing and the exchange rate.
- When `claimUnstakedToVault()` runs, `totalReservedLstWithdrawalHype` is decremented in the same EVM transaction, but the `sendAsset` to the `StakingVault` settles asynchronously on Core; until then the moved HYPE is reflected neither in the relay's delegator summary nor in the `StakingVault`'s `spotAccountBalance()`, deflating backing.

Because `exchangeRate()` and `totalBalance()` both depend on `stakingAccountBalance()`, any user mint/withdraw that lands inside one of these windows is mispriced. The same async-vs-counter mismatch also affects `getTotalStakedHype()`, which is used as a `Math.max` floor on backing.

Recommendations:

Account for in-flight state explicitly. Track the unstake-amount-in-flight separately from already-completed reserves and only add it once (either through `delegated` or through

`totalReservedLstWithdrawalHype`, not both). For `claimUnstakedToVault()`, mirror the EVM-side decrement of `totalReservedLstWithdrawalHype` with an "outbound to vault" counter that compensates `spotAccountBalance()` until the Core transfer is observable. Audit every consumer of `stakingAccountBalance()` and `getTotalStakedHype()` to ensure they remain consistent under these transitions.

As an alternative, mark the relevant accounting as "stale" whenever a Core-bound action mutates it (`withdrawStake()`, `claimUnstakedToVault()`, etc.) by stamping the current L1 block, and require at least a one L1 block gap before `stakingAccountBalance()/getTotalStakedHype()` can be read again. This mirrors the existing `lastDelegationChangeBlockNumber/lastSpotBalanceChangeBlockNumber` pattern and lets the precompile state catch up before any consumer derives the exchange rate from it.

HyperLink: Resolved with [PR-57](#).

Zenith: Verified.

[H-3] `withdrawStake()` does not validate the validator's available delegation

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`withdrawStake()` performs `CoreWriterLib.delegateToken(validator, coreAmount, true)` followed by `CoreWriterLib.withdrawStake(coreAmount)` without verifying that the relay has an active delegation to validator of at least `coreAmount`, nor that the delegation is past its `lockedUntilTimestamp`. The only guard is `summary.nPendingWithdrawals < 5`, which checks the global pending withdrawal count.

If the requested amount exceeds the live delegation to validator or the delegation is still locked (e.g. recently delegated or redelegated), the Core action will silently fail (CoreWriter actions do not revert on the EVM side), but `totalReservedLstWithdrawalHype` is still incremented by `_hypeCoreWeiToEvm(coreAmount)`. From the EVM accounting perspective the HYPE is reserved for a withdrawal that never occurs on Core, leaving the relay short and breaking solvency invariants used by `claimUnstakedToVault()` and the LST exchange rate. Subsequent claims will revert, and the StakingVault may become unable to honour user withdrawals despite an apparently sufficient reserve.

Recommendations:

Read the per-validator delegation via the `Delegations` precompile before calling `delegateToken/withdrawStake` and require `delegation.amount ≥ coreAmount` and `delegation.lockedUntilTimestamp ≤ block.timestamp`. Revert otherwise so that `totalReservedLstWithdrawalHype` is never incremented for an unstake that cannot be honoured by Core.

HyperLink: Resolved with [PR-59](#).

Zenith: Verified.

4.2 Medium Risk

A total of 5 medium risk findings were identified.

[M-1] The unstaking amount can become smaller due to rounding

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

When calculating unstaking amounts, the value is rounded down.

```
function finalizeBatch()
    external onlyOperator whenNotPaused whenBatchProcessingNotPaused {
        if (availableWithdrawalLiquidity < neededForWithdraws) {
            uint256 amountToUnstake = neededForWithdraws
            - availableWithdrawalLiquidity;
        }
        uint64 unstakeWei = amountToUnstake.to8Decimals();

        // Hyperliquid unstake enters the 7-day withdrawal queue; it never
        // credits vault spot in this call.
        // Unstake first so the same-block re-stake path remains valid.
        if (unstakeWei > 0) {
            (bool hasDelegation, PrecompileLib.Delegation memory delegation)
            = stakingVault.delegation(validator);
            require(hasDelegation && delegation.amount >= unstakeWei,
                NotEnoughBalance());
            stakingVault.unstake(validator, unstakeWei);
        }
    }
```

For example, if the EVM amount is $1e18 + 100$, the corresponding Core wei amount becomes 1e8. As a result, totalReservedLstWithdrawalHype increases by only 1e18 instead

of $1e18 + 100$.

```
function withdrawStake(uint128 amt, address validator)
    external onlyStakingVault {
        $.totalReservedLstWithdrawalHype += _hypeCoreWeiToEvm(coreAmount);

        emit StakeWithdrawn(coreAmount, msg.sender);
    }
```

Later, when users claim their withdrawal, the check below may prevent the claim.

```
function claimUnstakedToVault(uint64 weiAmount)
    external onlyStakingVault whenNotPaused {
        uint256 amountEvm = _hypeCoreWeiToEvm(weiAmount);
        $.totalStakedHype = amountEvm >= $.totalStakedHype ? 0 :
        $.totalStakedHype - amountEvm;
        // Strict revert (no zero-clamp): silent clamping would mask drift
        between reserved and on-chain state.
        ▷ if (amountEvm > $.totalReservedLstWithdrawalHype)
            revert Errors.ClaimExceedsReservedLst();
        $.totalReservedLstWithdrawalHype -= amountEvm;
```

Recommendations:

The calculation could instead be rounded up. Alternatively, the withdrawal amount could be rounded down when users claim their withdrawal.

HyperLink: Resolved with [@1bbf60216b...](#)

Zenith: Verified.

[M-2] `verifyOrSyncTotalStakedHype()` can desync `totalStakedHype` from `totalReservedLstWithdrawalHype`

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`verifyOrSyncTotalStakedHype()` can be called by an admin with `sync = true` to overwrite `totalStakedHype` with:

```
actual = _hypeCoreWeiToEvm(s.delegated + s.undelegated +  
    s.totalPendingWithdrawal)
```

In `claimUnstakedToVault()`, `totalStakedHype` and `totalReservedLstWithdrawalHype` are decremented by the same amountEvm, encoding an invariant that the "reserved" portion of LST is always a sub-component of the tracked staked total. The sync, however, derives actual from `s.totalPendingWithdrawal`, which represents Core-side withdrawals still inside the 7-day lock and explicitly excludes balance that has already matured into the relay's spot account.

`totalReservedLstWithdrawalHype`, by contrast, includes both locked and matured-but-unclaimed reserves. After a sync, `totalStakedHype` may therefore drop below `totalReservedLstWithdrawalHype`, breaking the implicit ordering and causing `claimUnstakedToVault()` to clamp `totalStakedHype` to zero while leaving `totalReservedLstWithdrawalHype` non-zero. This corrupts the LST backing accounting consumed by `stakingAccountBalance()` and the `Math.max` floor in `StakingVaultManager`.

Recommendations:

Either include matured-but-unclaimed reserves in the actual computation (e.g. add the relay's spot HYPE balance attributable to LST claims), or sync both `totalStakedHype` and `totalReservedLstWithdrawalHype` together in a single admin action so the invariant `totalStakedHype ≥ totalReservedLstWithdrawalHype` is preserved.

HyperLink: Resolved with [PR-61](#).

Zenith: Verified.

[M-3] The token relay should only be allowed to delegate to the manager's validator

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

There is no validation in the `retryStakeFromSpot` function, allowing the admin to delegate spot HYPE to any validator.

```
function retryStakeFromSpot(uint64 weiAmount, address validator)
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
    whenNotPaused
{
    if (weiAmount == 0) revert Errors.ZeroOperationAmount();
    if (validator == address(0)) revert Errors.InvalidAddress();

    _requireCoreSpotBalance(HLConstants.hypeTokenIndex(), weiAmount, true);
    _stakeFromRelaySpot(weiAmount, validator);

    emit RetryStake(weiAmount, validator);
}
```

If some HYPE is delegated to a validator other than the manager's validator, switching to a new validator can become permanently impossible.

During validator switching, the redelegation amount includes the HYPE delegated to the non-manager validator.

```
function switchValidator(address newValidator) external onlyOwner {
    PrecompileLib.DelegatorSummary memory delegatorSummary
    = PrecompileLib.delegatorSummary(address(tokenRelay));
    if (delegatorSummary.delegated > 0) {
        > stakingVault.tokenRedelegate(validator, newValidator,
```

```
delegatorSummary.delegated);  
}  
}
```

However, in the `tokenRedelegate` function, this amount is only compared against the amount delegated to the manager's validator, causing the transaction to revert.

```
function tokenRedelegate(address fromValidator, address toValidator,  
    uint64 weiAmount)  
    external  
    onlyStakingVault  
    whenNotPaused  
{  
    for (uint256 i = 0; i < delegations.length; i++) {  
        if (delegations[i].validator == fromValidator) {  
            found = true;  
            if (delegations[i].amount < weiAmount)  
                revert Errors.InsufficientBalance();  
            if (delegations[i].lockedUntilTimestamp > block.timestamp  
                * 1000) {  
                revert Errors.StakeLocked();  
            }  
            break;  
        }  
    }  
}
```

As a result, switching to a new validator becomes impossible, and there is no way to recover or reuse the HYPE delegated to the incorrect validator.

Recommendations:

The `retryStakeFromSpot` function should only allow staking to the manager's validator.

HyperLink: Resolved with [@add9d0806c ...](#)

Zenith: Verified.

[M-4] The `verifyOrSyncTotalStakedHype` and `stakeFromSpot` functions should not be called within the same block

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

The `verifyOrSyncTotalStakedHype` function can be called by the owner to synchronize the `totalStakedHype` value. However, if it is called in the same block after `stakeFromSpot`, it can introduce an accounting inconsistency.

For example, assume the current delegated amount is 1000, there are no pending withdrawals, and `totalStakedHype` is 950, so the admin intends to sync it to 1000. Before the sync occurs, a new staking operation of 200 is performed through `stakeFromSpot`, increasing `totalStakedHype` to 1150.

```
function stakeFromSpot(uint64 weiAmount, address validator)
    external onlyStakingVault {
    _stakeFromRelaySpot(weiAmount, validator);

    uint256 amountEvm = _hypeCoreWeiToEvm(weiAmount);
    $.totalStakedHype += amountEvm;
}
```

However, the delegated amount remains 1000 within the same block because the delegation update has not yet been reflected. As a result, `verifyOrSyncTotalStakedHype` synchronizes `totalStakedHype` back to 1000, causing the manager's `totalBalance` to become incorrect and leading to further accounting issues.

```
function verifyOrSyncTotalStakedHype(bool sync, string calldata reason)
    external
    returns (bool healthy, int256 delta)
{
    if (sync) _checkRole(DEFAULT_ADMIN_ROLE);
    if (sync) {
```

```
        $.totalStakedHype = actual;  
        emit TotalStakedHypeReconciled(tracked, actual, reason);  
    }  
}
```

If the synchronization occurs in a later block, totalStakedHype will be correctly updated to 1200.

Recommendations:

```
function verifyOrSyncTotalStakedHype(bool sync, string calldata reason)  
external  
returns (bool healthy, int256 delta)  
{  
    if (block.number ≤ _getHyperlinkTokenRelayStorage()  
        .lastSpotBalanceChangeBlockNumber) {  
        revert Errors.StaleSpotBalance();  
    }  
  
    if (sync) _checkRole(DEFAULT_ADMIN_ROLE);  
    if (sync) {  
        $.totalStakedHype = actual;  
        emit TotalStakedHypeReconciled(tracked, actual, reason);  
    }  
}
```

HyperLink: Resolved with [@5aa30e6dae ...](#)

Zenith: Verified.

[M-5] Stale shadow staking balance can overstate the LST exchange rate

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)
- [HyperlinkTokenRelay.sol](#)

Description:

`stakingAccountBalance()` returns the maximum between the live backing derived from the `DelegatorSummary` precompile and the EVM-tracked `tokenRelay.getTotalStakedHype()` shadow counter:

```
return Math.max(backing, tokenRelay.getTotalStakedHype());
```

`getTotalStakedHype()` is an internally maintained counter that only changes when LST flows pass through the relay (deposits, `withdrawStake()`, `claimUnstakedToVault()`, or administrative reconciliation through `verifyOrSyncTotalStakedHype()`). It is not authoritative HyperCore state. If actual backing falls because of validator slashing, a failed or no-op Core staking action, or other accounting drift, the live delegated + undelegated + `totalReservedLstWithdrawalHype` term decreases while `totalStakedHype` can remain at the stale higher value.

Because the `Math.max` uses the shadow counter as a pricing floor, `totalBalance()` and `exchangeRate()` continue reporting an exchange rate based on the stale balance until the counter is reconciled. Withdrawal batches finalized in this window snapshot the inflated exchange rate and later redeem `hHYPE` using it. Fast withdrawers can therefore receive more `HYPE` than their pro-rata share of the real backing, shifting the loss to remaining `hHYPE` holders. Depositors during the same drift window can also be under-minted because their deposits are priced against overstated assets.

Recommendations:

Review and reconsider the use of `Math.max(backing, tokenRelay.getTotalStakedHype())` in `stakingAccountBalance()`. Using `getTotalStakedHype()` as a floor masks legitimate decreases in the validator-side balance (slashes, drift) and lets the exchange rate stay

inflated until the relay-tracked counter is realigned. Consider using the authoritative backing for share pricing, and treat `totalStakedHype` as a monitoring and reconciliation signal. If the shadow value is required to mitigate transient precompile staleness during in-flight Core operations, restrict it to that scenario and revert or pause deposits and withdrawal processing whenever `verifyOrSyncTotalStakedHype(false, ...)` reports unhealthy drift or backing falls below `getTotalStakedHype()` by more than an expected tolerance.

HyperLink: Resolved with [PR-72](#).

Zenith: Verified. The `max(backing, tokenRelay.getTotalStakedHype())` has been removed.

4.3 Low Risk

A total of 6 low risk findings were identified.

[L-1] `_validateAccess()` has unclear and inconsistent semantics

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`_validateAccess(token, user)` is the single chokepoint for the token allowlist and user blacklist/whitelist checks, but its behaviour is hard to reason about:

- The `address(registry) = address(0)` branch is effectively dead code: `setInfrastructure()` and the initializer reject zero addresses, so the registry is always set in normal operation. The fallback path silently allows `token = address(0)` and rejects everything else, which contradicts the rest of the contract that uses `NATIVE_HYPE` (a non-zero sentinel) to identify HYPE.
- `token = address(0)` is implicitly allowed (skipping `isTokenAllowed`) instead of being mapped to a known token. If the intent was "always allow HYPE without consulting the registry", the constant should be `NATIVE_HYPE`, not `address(0)`.
- `user = address(0)` skips both `isUserBlacklisted` and `canUserInteract`. There is no clear use case for a zero user, and bypassing the access checks for it can easily mask a missing input validation upstream.

The inconsistency makes it possible for a future caller to pass `address(0)` (for either argument) and unknowingly bypass intended controls.

Recommendations:

Remove the dead `registry = address(0)` branch (or replace it with a hard revert), validate `token ≠ address(0)` and `user ≠ address(0)` at the entry of `_validateAccess()`, and use `NATIVE_HYPE` consistently if the intent is to always allow native HYPE without consulting the registry.

HyperLink: Resolved with [PR-66](#).

Zenith: Verified.

[L-2] `withdrawWithAuthorization()` quotes fees against the net amount instead of the gross

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [HyperlinkTokenRelay.sol](#)

Description:

In `withdrawWithAuthorization()`, the cap on `auth.feeAmount` is computed as:

```
uint256 maxFeeAmount = _quoteWithdrawalFee($, tokenIndex, auth.amount);
```

`auth.amount` is the net amount delivered to the recipient (the fee is charged separately via `feeAmount` and disbursed to the `FeeManager`). `_quoteWithdrawalFee()` delegates to `feeManager.calculateFee(amount, FeeType.WITHDRAWAL)`, which interprets its input as the gross amount the fee is being taken out of. Passing the net to a function that expects the gross under-quotes the maximum allowed fee, so the cap never matches the `FeeManager`'s own definition of the fee for the same withdrawal.

In practice, the enclave can still produce a `feeAmount` that satisfies the relay's check, but offline tooling using `quoteWithdrawalFee()` to estimate fees will be inconsistent with what the `FeeManager` would compute on a gross amount, and any future use of the same helper outside the enclave-signed path will be subtly wrong.

Recommendations:

Quote the fee against the gross amount: pass `auth.amount + auth.feeAmount` (or the gross value the enclave used) into `_quoteWithdrawalFee()`, and document the convention so the enclave, the relay and the `FeeManager` agree on what the "amount" parameter represents.

HyperLink: Acknowledged. Not fixing - fees are intended to be based on the recipient amount, not total debit. The enclave calculates the fee, reserves `amount + fee`, and signs both values. `_quoteWithdrawalFee(auth.amount)` just caps the signed fee against that same amount.

[L-3] The exchange rate should decrease when a slashing is applied

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

When applying slashing, the slashed exchange rate is only validated to ensure it is lower than the snapshotted exchange rate.

```
function applySlash(uint256 batchIndex, uint256 slashedExchangeRate)
    external onlyOwner {
        // Slashed rate must not exceed the snapshot rate (slashing can only
        // reduce value)
        require(slashedExchangeRate <= batch.snapshotExchangeRate,
            InvalidBatch(batchIndex));
    }
```

However, if another slashing is applied afterward, the new slashed exchange rate may become higher than the previous slashed rate.

This can cause issues in the `setMinimumStakeBalance` function. If `setMinimumStakeBalance` is called after the first slashing, the `minimumStakeBalance` is calculated using the older, lower exchange rate.

```
function setMinimumStakeBalance(uint256 _minimumStakeBalance)
    public onlyOwner {
        require(totalBalance() >= _minimumStakeBalance,
            MinimumStakeBalanceTooLarge());
        if (currentBatchIndex < batches.length) {
            uint256 newWithdrawCapacity = totalBalance() - _minimumStakeBalance;
            StakingVaultManager.Batch memory batch = batches[currentBatchIndex];
            uint256 _exchangeRate = batch.slashed ? batch.slashedExchangeRate :
            batch.snapshotExchangeRate;
            require(
                newWithdrawCapacity >= _h1HYPETOHYPER(batch.h1hypeProcessed,
```

```
        _exchangeRate),
        MinimumStakeBalanceTooLarge()
    );
}
minimumStakeBalance = _minimumStakeBalance;
emit MinimumStakeBalanceUpdated(_minimumStakeBalance);
}
```

Later, if a new slashing is applied with a higher exchange rate, the batch may no longer be finalizable because the `minimumStakeBalance` was computed using the previous lower rate.

Recommendations:

```
function applySlash(uint256 batchIndex, uint256 slashedExchangeRate)
    external onlyOwner {
    // Slashed rate must not exceed the snapshot rate (slashing can only
    // reduce value)
    require(slashedExchangeRate ≤
        batch.snapshotExchangeRate, InvalidBatch(batchIndex));

    uint256 oldExchangeRate = batch.slashed ? batch.slashedExchangeRate :
        batch.snapshotExchangeRate;

    require(slashedExchangeRate ≤
        oldExchangeRate, InvalidBatch(batchIndex));
```

HyperLink: Resolved with [@c55d5dbfd ...](#)

Zenith: Verified.

[L-4] There should be a fallback mechanism in case switching to a new validator fails

SEVERITY: Low

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

In some cases, redelegation to a new validator can fail due to insufficient validation of the new validator.

```
function tokenRedelegate(address fromValidator, address toValidator,
    uint64 weiAmount)
    external
    onlyStakingVault
    whenNotPaused
{
    CoreWriterLib.delegateToken(fromValidator, weiAmount, true);
    CoreWriterLib.delegateToken(toValidator, weiAmount, false);
}
```

As a result, all HYPE delegated to the old validator becomes undelegated, and there is currently no way to reuse or redelegate those funds.

Recommendations:

Add an admin operation to recover or reuse the undelegated amounts from the old validator during the such emergency situation. Alternatively, ensure that redelegation to a new validator always succeeds by checking the new validator carefully.

HyperLink: Resolved with [@efb519763a ...](#)

Zenith: Verified.

[L-5] The system should not be able to unpause while in emergency mode

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

When emergency mode is scheduled, the system is automatically paused, preventing operators from blacklisting users. However, there is no restriction preventing the system from being unpaused during this period.

```
function setEmergencyMode(bool enabled)
    external onlyRole(EMERGENCY_MANAGER_ROLE) {
    if (enabled) {
        if (scheduledTime == 0) {
            // Freeze the latest published exit snapshot before the timelock
            starts.
            uint256 snapshotEpoch = $.stateManager.lastClaimRootEpoch();
            if (snapshotEpoch == 0) revert Errors.NoClaimRoot();

            scheduledTime = block.timestamp + EMERGENCY_MODE_TIMELOCK;
            $.emergencyModeScheduledTime = scheduledTime;
            $.emergencySnapshotEpoch = snapshotEpoch;

            // Atomic cascade so OPERATOR can't blacklist users mid-timelock.
            _pauseSystem(true);
```

Recommendations:

Prevent the system from being unpaused during emergency mode, and also restrict changes to critical system contracts such as the vault and feeManager.

HyperLink: Resolved with [@5470002d41 ...](#)

Zenith: Verified.

[L-6] Mutable claim buffer can reopen the slash window after withdrawals are claimed

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

StakingVaultManager uses the current `claimWindowBuffer` value in both the claimability check and the slash-window check:

```
block.timestamp >= batch.finalizedAt + 7 days + claimWindowBuffer
```

and:

```
block.timestamp < batch.finalizedAt + 7 days + claimWindowBuffer
```

This makes the boundary mutable after a batch has already been finalized. If the owner shortens `claimWindowBuffer`, users can become claimable earlier and increment `totalHypeClaimed` using the batch exchange rate. If the owner later extends `claimWindowBuffer`, the same finalized batch can become slashable again even though claims have already started.

Applying a slash after claims have started can reduce `totalHypeProcessed` below `totalHypeClaimed`:

```
totalHypeProcessed -= _h1HYPETOHYPE(batch.h1hypeProcessed, oldExchangeRate);  
totalHypeProcessed += _h1HYPETOHYPE(batch.h1hypeProcessed,  
    slashedExchangeRate);
```

Several paths then compute `totalHypeProcessed - totalHypeClaimed`, including `totalBalance()` and `_unstakableLiquidity()`. If the processed total is reduced below the already-claimed total, these reads revert with a Solidity underflow and can DoS deposits, batch processing, finalization, and other accounting-dependent operations.

The issue requires privileged configuration changes and a privileged slash operation, so it

is unlikely to happen accidentally in normal use. However, the accounting invariant should not depend on a mutable global buffer once users have begun claiming a finalized batch.

Recommendations:

Prevent slashing once claims have started for a batch, or snapshot the slash/claim boundary per batch at finalization so later `claimWindowBuffer` changes cannot reopen the slash window.

HyperLink: Resolved with [PR-73](#).

Zenith: Verified.

4.4 Informational

A total of 17 informational findings were identified.

[I-1] There is an unnecessary conditional check in `setMinimumStakeBalance` function

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

If `currentBatchIndex` is less than the length of the `batches` array, it means the current batch has not been finalized yet. Therefore, slashing cannot be applied to that batch, and `snapshotExchangeRate` can always be used below.

```
function setMinimumStakeBalance(uint256 _minimumStakeBalance)
    public onlyOwner {
        require(totalBalance() >= _minimumStakeBalance,
            MinimumStakeBalanceTooLarge());
        if (currentBatchIndex < batches.length) {
            uint256 newWithdrawCapacity = totalBalance() - _minimumStakeBalance;
            StakingVaultManager.Batch memory batch = batches[currentBatchIndex];
            ▶ uint256 _exchangeRate = batch.slashed ? batch.slashedExchangeRate :
                batch.snapshotExchangeRate;
            require(
                newWithdrawCapacity >= _h1HYPEToHYPE(batch.h1hypeProcessed,
                    _exchangeRate),
                MinimumStakeBalanceTooLarge()
            );
        }
        minimumStakeBalance = _minimumStakeBalance;
        emit MinimumStakeBalanceUpdated(_minimumStakeBalance);
    }
```

Recommendations:

```
function setMinimumStakeBalance(uint256 _minimumStakeBalance)
    public onlyOwner {
        require(totalBalance() >= _minimumStakeBalance,
            MinimumStakeBalanceTooLarge());
        if (currentBatchIndex < batches.length) {
            uint256 newWithdrawCapacity = totalBalance() - _minimumStakeBalance;
            StakingVaultManager.Batch memory batch = batches[currentBatchIndex];

            uint256 _exchangeRate = batch.slashed ? batch.slashedExchangeRate :
                batch.snapshotExchangeRate;
            require(
                newWithdrawCapacity ≥ _hlHYPEToHYPE(batch.hlhypeProcessed, _
                    exchangeRate),
                newWithdrawCapacity ≥ _hlHYPEToHYPE(batch.hlhypeProcessed,
                    batch.snapshotExchangeRate),
                MinimumStakeBalanceTooLarge()
            );
        }
        minimumStakeBalance = _minimumStakeBalance;
        emit MinimumStakeBalanceUpdated(_minimumStakeBalance);
    }
```

HyperLink: Resolved with [@4c9fcc2bb6 ...](#)

Zenith: Verified.

[I-2] There is a incorrect comment in emergencyStakingWithdraw function

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

Below comment is incorrect in emergencyStakingWithdraw function.

```
function emergencyStakingWithdraw(uint256 amount, string calldata purpose)
    external onlyOwner {
        // Immediately undelegate HYPE
        // Queue a staking withdrawal, subject to the 7-day withdrawal queue.
        Amount will be available in
        ▷ // the StakingVault's spot account balance after 7 days.
        stakingVault.unstake validator, amount.to8Decimals();
        emit EmergencyStakingWithdraw(msg.sender, amount, purpose);
    }
```

Recommendations:

```
function emergencyStakingWithdraw(uint256 amount, string calldata purpose)
    external onlyOwner {
        // Immediately undelegate HYPE
        // Queue a staking withdrawal, subject to the 7-day withdrawal queue.
        Amount will be available in
        // the StakingVault's spot account balance after 7 days.
        // the TokenRelay's spot account balance after 7 days.
        stakingVault.unstake validator, amount.to8Decimals();
        emit EmergencyStakingWithdraw(msg.sender, amount, purpose);
    }
```

HyperLink: Resolved with [@49c0677e3b ...](#)

Zenith: Verified.

[I-3] The exchange rate can change significantly when the admin reconciles totalReservedLstWithdrawalHype

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

When the admin calls the `reconcileTotalReservedLstWithdrawalHype` function, the manager's `totalBalance` changes immediately, which can cause the exchange rate to change.

```
function reconcileTotalReservedLstWithdrawalHype(uint256 newAmount,
    bytes32 reason)
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    HyperlinkTokenRelayStorage storage $ = _getHyperlinkTokenRelayStorage();
    uint256 oldAmount = $.totalReservedLstWithdrawalHype;
    $.totalReservedLstWithdrawalHype = newAmount;
    emit ReservedLstWithdrawalHypeReconciled(oldAmount, newAmount, reason);
}
```

Recommendations:

Should acknowledge this.

HyperLink: Resolved with [@f32f54e598 ...](#)

Zenith: Verified.

[I-4] There is an unnecessary check in the deposit function

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

In the deposit function, if the converted H1HYPE amount is greater than 0, then the input HYPE amount must also already be greater than 0. Therefore, the check below is unnecessary.

```
function deposit() external payable whenNotPaused {
    // IMPORTANT: Calculate shares _before_ transferring HYPE to vault,
    // otherwise the exchange rate would include the new deposit
    uint256 amountToMint = HYPETO1H1HYPE(amountToDeposit);
    require(amountToMint > 0, ZeroAmount());
    h1HYPE.mint(msg.sender, amountToMint);

    // Transfer HYPE to staking vault (HyperEVM → HyperEVM)
    > if (amountToDeposit > 0) {
        stakingVault.deposit{value: amountToDeposit}();
    }
}
```

Recommendations:

```
function deposit() external payable whenNotPaused {
    // IMPORTANT: Calculate shares _before_ transferring HYPE to vault,
    // otherwise the exchange rate would include the new deposit
    uint256 amountToMint = HYPETO1H1HYPE(amountToDeposit);
    require(amountToMint > 0, ZeroAmount());
    h1HYPE.mint(msg.sender, amountToMint);

    // Transfer HYPE to staking vault (HyperEVM → HyperEVM)
```

```
if (amountToDeposit > 0) {  
    stakingVault.deposit{value: amountToDeposit}();  
}
```

HyperLink: Resolved with [@9096e71ad0 ...](#)

Zenith: Verified.

[I-5] The minimum withdrawal amount can be calculated using the exchange rate of the current batch

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

In the `queueWithdraw` function, the minimum withdrawal amount check uses the current exchange rate.

```
function queueWithdraw(uint256 hlhypeAmount)
    external whenNotPaused returns (uint256[] memory) {
    require(hlhypeAmount > 0, ZeroAmount());
    require(hlHYPEToHYPE(hlhypeAmount) >= minimumWithdrawAmount,
        BelowMinimumWithdrawAmount());
```

However, these withdrawals are processed using the current batch's exchange rate instead.

Recommendations:

Update the logic to use the current batch's exchange rate, or explicitly document that the current exchange rate is intentionally used here.

HyperLink: Resolved with [@b9e29ea5b4 ...](#)

Zenith: Verified.

[I-6] There is an unnecessary check in the processBatch function

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

When a `withdrawal` is cancelled, it is removed from the queue.

```
function cancelWithdraw(uint256 withdrawalId) external whenNotPaused {  
    // Remove from the linked list  
    withdrawalQueue.remove(withdrawalId);  
  
    // Set cancelled timestamp  
    withdrawal.cancelledAt = block.timestamp;  
}
```

This means all withdrawals remaining in the queue are active, and their `cancelledAt` value is always 0. Therefore, the check below is unnecessary.

```
function processBatch(uint256 numWithdrawals)  
    public whenNotPaused whenBatchProcessingNotPaused returns (uint256) {  
    while (withdrawCapacityAvailable > 0 && numWithdrawalsLeft > 0) {  
        // Get the next withdrawal to process  
        (bool hasNextNode, uint256 nextNodeId)  
        = withdrawalQueue.getNextNode(lastProcessedWithdrawId);  
        if (!hasNextNode) {  
            break;  
        }  
  
        Withdraw storage withdraw = withdrawals[nextNodeId];  
  
        // Update batch metrics (in memory)  
        if (withdraw.cancelledAt == 0) {  
            batch.h1hypeProcessed += withdraw.h1hypeAmount;  
        }  
    }  
}
```

```
    }  
  }  
}
```

Recommendations:

```
function processBatch(uint256 numWithdrawals)  
  public whenNotPaused whenBatchProcessingNotPaused returns (uint256) {  
    while (withdrawCapacityAvailable > 0 && numWithdrawalsLeft > 0) {  
      // Get the next withdraw to process  
      (bool hasNextNode, uint256 nextNodeId)  
    = withdrawQueue.getNextNode(lastProcessedWithdrawId);  
      if (!hasNextNode) {  
        break;  
      }  
  
      Withdraw storage withdraw = withdraws[nextNodeId];  
  
      // Update batch metrics (in memory)  
      if (withdraw.cancelledAt == 0) {  
        batch.hlHypeProcessed += withdraw.hlHypeAmount;  
      }  
    }  
  }  
}
```

HyperLink: Resolved with [@9b031ca2b5 ...](#)

Zenith: Verified.

[I-7] Dust accounting overflow check in `_disburseFromCore()` is overly conservative

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

The dust-tracking branch in `_disburseFromCore()` is gated on `netAmount ≤ type(uint128).max` & `feeAmount ≤ type(uint128).max`. The actual concern is that `netAmount + feeAmount` does not overflow `uint256` before being passed to `HLConversions.evmToWei`. Checking each operand individually against `uint128` is both stricter than needed and harder to read.

Recommendations:

Express the precondition directly, e.g. `if (netAmount ≤ type(uint256).max - feeAmount)`, or simply rely on Solidity 0.8 checked arithmetic in the addition and remove the manual guard.

HyperLink: Resolved with [PR-74](#).

Zenith: Verified.

[I-8] Withdrawal recipient is not checked against the protocol registry

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`withdrawWithAuthorization()` validates registry access for `auth.user` through `_validateAccess(auth.token, auth.user)`, but the withdrawal recipient is not checked against the protocol registry. After the enclave authorization is validated, `auth.to` only needs to be nonzero, not be the token system address, and have a valid HyperCore account.

If the protocol registry is intended to restrict all parties that can participate in protocol transfers, an allowed user can withdraw to a blacklisted or otherwise non-whitelisted recipient. Although the enclave signature commits to `auth.to`, the contract's on-chain registry enforcement applies only to the debited user, leaving recipient policy dependent on off-chain signing behavior.

Recommendations:

If registry restrictions should apply to withdrawal recipients, validate `auth.to` on-chain when it can differ from `auth.user`:

```
_validateAccess(auth.token, auth.user);  
if (auth.to != auth.user) {  
    _validateAccess(auth.token, auth.to);  
}
```

If withdrawals to unrestricted recipients are intentional, document that protocol registry checks apply only to the account whose enclave balance is debited, not to the recipient.

HyperLink: Resolved with [PR-84](#).

Zenith: Verified.

[I-9] addValidator() and removeValidator() do not emit events

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVault.sol](#)

Description:

addValidator() mutates the whitelistedValidators mapping and removeValidator() deletes the entry from it, but neither function emits an event. Off-chain monitoring and indexers cannot reliably track validator allowlist changes without scanning storage.

Recommendations:

Emit dedicated events (e.g. ValidatorAdded(address validator) and ValidatorRemoved(address validator)) on each successful insertion and removal.

HyperLink: Resolved with [PR-50](#).

Zenith: Verified.

[I-10] Redundant whenNotPaused modifier on HLHYPE.mint()/burn()

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HLHYPE.sol](#)

Description:

`mint()` and `burn()` both carry the `whenNotPaused` modifier, but they delegate to OpenZeppelin's `_mint/_burn`, which call `_update`. `_update` is already overridden with `whenNotPaused`, so the pause check runs twice on every `mint/burn`.

Recommendations:

Drop the `whenNotPaused` modifier from `mint()` and `burn()` and rely on the `_update` override as the single enforcement point.

HyperLink: Resolved with [PR-75](#).

Zenith: Verified.

[\[I-11\] transferHypeToCore\(\) can lose dust by not truncating to 8 decimals](#)

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVault.sol](#)

Description:

`transferHypeToCore()` forwards the full amount (18 decimals) to `HYPE_SYSTEM_ADDRESS` via `_transfer()`, while the bridge to Core only credits 8-decimal precision. Any sub-`HYPE_CORE_TO_EVM_SCALE` (`1e10`) wei in amount is sent on the EVM side but lost when the bridge truncates, and the spot delta recorded via `amount.to8Decimals()` already discards those low-order digits.

Recommendations:

Truncate amount to 8-decimal precision before calling `_transfer()` (e.g. `uint256 amount8 = amount.to8Decimals().to18Decimals()`), or revert if `amount % HYPE_CORE_TO_EVM_SCALE  $\neq$  0`, so dust is either kept in the EVM balance or rejected explicitly.

HyperLink: Resolved with [PR-76](#).

Zenith: Verified.

[I-12] HYPE token id is duplicated instead of using `HLConstants.hypeTokenIndex()`

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVault.sol](#)

Description:

`StakingVault.HYPE_TOKEN_ID` is set via the constructor argument and stored as an immutable, while the rest of the codebase (notably `HyperlinkTokenRelay`) reads the same value through `HLConstants.hypeTokenIndex()`. Maintaining two sources of truth for the HYPE token index increases the risk of misconfiguration at deployment time (mainnet vs testnet) and makes future migrations more error prone.

Recommendations:

Use `HLConstants.hypeTokenIndex()` consistently across both contracts, or derive `HYPE_TOKEN_ID` from the same constant so deployments cannot diverge.

HyperLink: Resolved with [PR-77](#).

Zenith: Verified.

[I-13] EVM accounting can diverge from Core state on async failures

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)
- [StakingVault.sol](#)

Description:

Several flows update EVM-side accounting in lockstep with an action scheduled on HyperCore (CoreWriter actions, sendAsset, bridging, stake/unstake, etc.). The EVM mutation is committed immediately when the transaction lands, but the corresponding Core action is processed asynchronously and can silently fail (CoreWriter actions do not surface failures back to the EVM). When this happens the persisted EVM state no longer reflects the true on-Core state.

The same async gap also affects the observable state read from EVM, which can become stale when transfers are queued on Core or assets are bridged: precompile reads (spot balances, delegator summary, etc.) reflect the pre-action snapshot until Core settles, so any consumer that combines the precompile view with EVM-side counters can momentarily see double-counted or under-counted balances.

This particularly affects `totalStakedHype` and `totalReservedLstWithdrawalHype` in `HyperlinkTokenRelay`, which are bumped when staking/unstaking is queued; if the Core-side `tokenDelegate/withdrawStake` fails, the variables remain inflated/deflated until manually reconciled.

Operators have `reconcileTotalReservedLstWithdrawalHype()` and `verifyOrSyncTotalStakedHype()` to manually correct drift, but these require off-chain monitoring and admin intervention; smaller discrepancies (e.g. transient `sendAsset` failures during emergency mode) may go unnoticed until they compound.

Recommendations:

Document every flow where EVM state advances ahead of Core settlement, and minimise the surface area where silent Core failures are possible (e.g. tighten preconditions before calling `CoreWriter` actions). Where feasible, defer the EVM mutation until the Core side is

observable (e.g. derive `totalReservedLstWithdrawalHype` from the precompile delegator summary instead of an internal counter). At minimum, expose explicit health-check views and events that off-chain monitors can use to detect divergence early, and require admin reconciliation hooks for every counter that can drift.

HyperLink: Resolved with [PR-86](#).

Zenith: Verified.

[I-14] usedAuthorizations mapping is redundant with the per-user nonce

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`_validateWithdrawalAuthorization()` requires `auth.nonce = $.nonces[auth.user]` and increments `$.nonces[auth.user]` on success. Because the nonce is part of the EIP-712 struct, any subsequent submission of the same auth will fail the nonce check, making the additional `$.usedAuthorizations[structHash]` mapping unnecessary for replay protection. Maintaining the mapping costs an extra SSTORE per withdrawal and an additional SLOAD on every retry.

Recommendations:

Drop the `usedAuthorizations` mapping and rely solely on the monotonically increasing per-user nonce for replay prevention.

HyperLink: Resolved with [PR-63](#).

Zenith: Verified. `usedAuthorizations` has been removed.

[\[I-15\] _setTargetPaused\(\) can revert during emergency actions](#)

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`_setTargetPaused()` calls `_requireContractAddr(target)` before forwarding to `IPauseTarget(target).setPaused(_paused)`. This helper is invoked from emergency-mode flows; a revert here (e.g. if a target is set to a non-contract address by mistake) prevents the operator from pausing the rest of the system in a critical situation. Validation should happen when the target reference is set, not when the emergency action is executed.

Recommendations:

Drop the `_requireContractAddr(target)` check from `_setTargetPaused()` and instead enforce `target.code.length > 0` in the setters that configure these references. Optionally swallow individual target failures (try/catch) so a single misconfigured target does not block the rest of the emergency response.

HyperLink: Resolved with [PR-64](#).

Zenith: Verified.

[I-16] `exitWithCommitment()` unnecessarily increments the withdrawal nonce

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`exitWithCommitment()` performs `$.nonces[user]++` after updating `highestExitNonce[user][token]`. Replay protection for exit commitments is already provided by `highestExitNonce`, and `$.nonces` is the nonce used by the regular withdrawal flow (`_validateWithdrawalAuthorization`). Bumping the withdrawal nonce from the emergency path can invalidate any in-flight `WithdrawalAuthorization` the enclave previously signed for the same user, with no security benefit.

Recommendations:

Remove the `$.nonces[user]++` increment from `exitWithCommitment()`; rely on `highestExitNonce[user][token]` to prevent commitment replays.

HyperLink: Resolved with [PR-65](#).

Zenith: Verified.

[I-17] `cumulativeExitWithdrawn` conflates normal and emergency withdrawals

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

`cumulativeExitWithdrawn[user][token]` is incremented by both `withdrawWithAuthorization()` and `exitWithCommitment()`. In `exitWithCommitment()`, the cap on the emergency payout is computed as `effectiveBalance - alreadyWithdrawn`, where `alreadyWithdrawn` includes every normal withdrawal the user ever made before emergency mode was activated.

For this cap to make sense, `effectiveBalance` (derived from the snapshot proof) must therefore represent the user's historical credit (lifetime deposits minus prior withdrawals plus accrued balance), not just the live balance at snapshot time. If the snapshot tree only encodes the active balance at the moment the snapshot was frozen, users who withdrew normally before the emergency will see their emergency-exit cap incorrectly reduced (or zeroed) by the historical `cumulativeExitWithdrawn`.

Recommendations:

Document precisely what `snapshotBalance` represents in the off-chain snapshot construction, and either ensure it includes prior withdrawn amounts (so the subtraction is correct) or split tracking into two counters: one for normal withdrawals and one for emergency exits, with `exitWithCommitment()` only consuming the latter.

HyperLink: Resolved with [PR-78](#).

Zenith: Verified.